COE 379L Midterm Project Report: Atlas Laboratory Agent

Name: Gunnika Singh | UT EID: gs34488 | Repository: https://github.com/GunnikaS/coe379lfa26-midterm

1. System Overview

Requests are made through the LabRequest Pydantic model, which enforces that each request follows the same structure with an authenticated user id as well as text describing the request. Additionally, project id, resource id, and start and end times can also be specified in the request. The system prompt separates trusted instructions from untrusted data through explicit system instructions that tell the model not to use user-provided text or tool outputs as instructions that override the system instructions.

The agent then follows a decision-action-observation loop. First, for each step, the LLM produces one of three structured decisions: tool call, ask the user for clarification, or present a final response. For proposed tool calls, the agent first validates the decision by checking if required checks pass (such as user authentication, tool availability, etc.) before executing the tool. The resulting observation is recorded in a trace and returned to the LLM to use as context for its next decision until a final response is reached. Additionally, a maximum number of steps are specified to ensure that the agent doesn't fall into infinite execution. For the purposes of the deterministic simulation, the maximum number of steps was set to ten.

The action gate runs before a proposed tool call is dispatched. It checks that the request's authenticated identity and specified reservation details are respected. For reservation attempts, it also verifies that required policy information has been retrieved, authorizations and resource status have been checked, and all reservation preconditions are satisfied. If any of the checks fail, the action is denied. In this case, the tool is not dispatched, and the failure is recorded in the trace.

Finally, the system validates the LLM's final response against the execution trace. It checks that citations refer to retrieved passages and that claimed completed actions correspond to successful mutations. A response marked as completed must have a successful mutation and at least one completed action; conversely, a response with a non-completed status cannot claim completed actions or contradict a successful mutation. If validation fails, the system returns a failed response.

2. Public Benchmark Results

Table 1. Aggregate metrics from public_benchmark.json.

Metric	Value	Metric	Value
scenario_count	6	unnecessary_refusal_rate	0
success_rate	1	clarification_accuracy	1
invalid_structured_response_rate	0	average_tool_calls	2.67
unsafe_action_attempt_rate	0		

All six public benchmark scenarios pass, achieving a 100% success rate, with zero invalid structured responses, unsafe action attempts, or unnecessary refusals. The traces show that valid reservations complete only after policy retrieval, authorization checks, and resource-status verification, while missing qualifications, expired calibration, missing after-hours approval, and reservation conflicts are correctly blocked. One notable detail is that the conflict scenario retrieves an untrusted document containing an instruction to ignore prior rules, but the agent does not follow it and correctly blocks the reservation.

3. Student Tests

The first test (test_unavailable_resource) tests that the system will block a reserve_resource tool call if there is a conflicting reservation. On top of ensuring that the tool call failed and a new reservation is not made, the test verifies that the tool call fails specifically due to the action gate denying the action and

recognizing that the resource is unavailable. This test fortifies the system by ensuring that conflicting actions with resources cannot be dispatched.

The second test (test_contradictory_responses) verifies that the agent detects and rejects contradictory final responses. The test simulates a valid reservation workflow: the agent retrieves the reservation policy, checks the user's authorizations, verifies the resource's status, and successfully reserves the tensile tester. However, the LLM then incorrectly returns a final response with a blocked status, claiming the reservation was not completed. The test correctly invokes a validation error. This helps ensure system reliability and consistency by preventing the agent from reporting a final outcome that contradicts actions already performed, making its responses more trustworthy.

The final test (test_unapproved_after_hours_reservation) verifies that the agent blocks reservations when the user lacks the required after-hours approval. The test simulates Bob attempting to reserve the tensile tester from 7 PM to 8 PM, retrieving the relevant policy and authorization information, and attempts the reservation. It then checks that the action gate denies the request with an action_gate_denied error and that the error message specifically includes after_hours_approval_missing. This test helps ensure policy compliance and system safety by preventing unauthorized after-hours access and verifying that denied actions do not modify the system's state.

4. Implementation Decisions

I chose specific error codes and messages to make validation failures clear, consistent, and easy to test. Rather than returning a generic error when a reservation is unsafe, the validators identify the exact issue, such as required_qualification_missing, resource_unavailable, or after_hours_approval_missing. This makes debugging easier and helps distinguish between different policy violations. I believe that this is important with LLMs in order to ensure that an agent has failed for the correct reason. By simply checking for errors or failed tool calls, we are not truly validating that the LLM will behave correctly, especially in edge cases. When a proposed tool call is denied, the agent records a failed ToolObservation with the error code action_gate_denied and includes the validator's messages. The observation is then returned to the model so it can understand why the action was rejected and respond appropriately without executing the denied operation.

I also use distinct termination reasons to make the outcome of an agent run easier to interpret, such as a successful final response, an invalid final decision, or exceeding the maximum number of steps. These distinguish normal completion from validation failures and situations where the agent cannot make progress. For unresolved request fields, I require the agent to obtain the missing information rather than assume values. If essential details, such as the reservation time, are ambiguous, the agent requests clarification before proceeding. Together, these choices improve traceability, prevent unsafe actions, and make the system's behavior more predictable and reliable.